# Development Plan, Proof of Concept Plan, and Team Charter Software Engineering

Team 23, Verity
Calvin Salsali
Raymond Tao
Jaden Moore
Richard Han

October 19, 2026

Table 1: Revision History

| Date | Developer(s) | Project and Development Plan Change |
|---|---|---|
| 2026-09-25 | Calvin Salsali, Raymond Tao, Jaden Moore, Richard Han | Defined initial team roles, Git/PR workflow, CI/CD strategy, Proof of Concept (POC) feasibility question, engineering tool evaluation, and Team Charter. |
| 2026-09-28 | Jaden Moore, Richard Han | Added coding standards (PEP 8 and TypeScript), issue priority definitions (`P0`–`P2`), and initial milestone schedule. |
| 2026-10-19 | Calvin Salsali, Raymond Tao, Jaden Moore, Richard Han | Addressed TA feedback (Issues #19–#21) and Peer Review Team 12 feedback (Issues #2, #3, #13): specified concrete candidate detectors, benchmark corpora, naive baselines, and +0.10 Macro-$F_1$ baseline delta for the POC; added proactive false-positive mitigations; added concrete calendar milestone dates; defined the 3-step Team Charter check-in protocol; and added individual member reflections and completed GenAI verification audits. |

This development plan records the team's current decisions for how Verity will be organized, built, reviewed, evaluated, and demonstrated. The plan is intended for team members and future contributors, so decisions are stated concretely enough to guide work while remaining changeable when evidence from the proof of concept, requirements work, or later testing justifies a better approach.

The project repository is github.com/richy1337/Verity.

# 1 Confidential Information?

Verity is not currently based on confidential industry information and the team does not require proprietary customer data. Development and evaluation will use public, appropriately licensed, synthetic, or team created material unless course staff explicitly approve another source. If confidential information is introduced later, the team will document the governing agreement before using that information in the repository or product.

# 2 IP to Protect

The project is not currently subject to a separate intellectual property agreement. Team authored code and documentation are intended to be publicly reviewable. Third party models, datasets, libraries, and services remain subject to their own licences and terms, and those constraints will be recorded before the associated dependency is adopted.

# 3 Copyright License

The project uses the MIT License for team authored source code unless a later dependency or course approved constraint requires a change. The licence is stored in the repository's `LICENSE` file. This choice supports open review, portfolio use, and future experimentation while keeping licensing obligations straightforward.

# 4 Team Meeting Plan

The team will hold one scheduled project meeting each week, normally 45 to 60 minutes, with an additional technical working session when an active milestone requires one. At least some meetings will be in person. The team will use available capstone lecture or tutorial time when appropriate rather than publishing a physical meeting location online.

The meeting chair will rotate. Before each scheduled meeting, the chair will open or update a GitHub issue containing the agenda. The meeting will cover current milestone status, completed work, blockers, open design decisions, issue assignment, and the next concrete commitments. Action items will be written into GitHub issues before the meeting ends.

The project does not currently depend on an external industry supervisor. Meetings with the course TA, instructor, or another approved advisor will be scheduled when required. Questions that materially affect scope, evaluation, or architecture will be documented so that advice can be traced to the decision it influenced.

# 5 Team Communication Plan

GitHub is the durable source of truth for project work. Issues track tasks, bugs, research questions, meeting actions, and documentation work. Pull requests are used for reviewable code and document changes. Important decisions that would otherwise be lost in chat are summarized in an issue, pull request, or meeting note.

A team group chat may be used for quick coordination, availability, and urgent questions, but work is not considered assigned until it is represented in the repository. Email is used when communication with course staff or an external contact requires a formal record.

# 6 Team Member Roles

The team will use overlapping roles rather than rigid silos. The assignments below are the proposed starting point for team review; responsibilities may shift as evidence and workload change.

**Calvin Salsali**
: **Primary:** Frontend and product integration, result presentation, and interaction design. **Secondary:** ML evaluation review, documentation quality assurance, and integration support.

**Raymond Tao**
: **Primary:** Detection and evaluation, benchmark design, model investigation, and failure analysis. **Secondary:** Backend review, experiment reproducibility, and documentation review.

**Jaden Moore**
: **Primary:** Backend and system integration, analysis API, and CI and deployment concerns. **Secondary:** Frontend review, testing infrastructure, and model integration support.

**Richard Han**
: **Primary:** Testing and evaluation infrastructure, reproducibility and data workflow, and privacy and reliability review. **Secondary:** Backend support, model integration, and quality assurance.

Project level responsibilities rotate each milestone. One member is meeting chair, one is issue curator and notetaker, and one is quality assurance reviewer. The quality assurance reviewer checks the relevant course checklist and Avenue rubric before a deliverable is considered complete. For code changes, the author may not be the only reviewer.

Roles are intentionally flexible. At the start of each major milestone, the team will review workload, risk, and learning needs. A member may change primary areas or pair with another member when the current assignment creates a bottleneck or when another person needs domain experience. No technical area is owned by only one person for the entire course.

# 7 Workflow Plan

This section defines Team 23's repository workflow for branching and pull request reviews, GitHub Issue tracking, and Continuous Integration / Continuous Delivery (CI/CD) automation.

## 7.1 Git and Pull Requests

The default branch is `main`. Work that is more than a trivial correction is completed on short lived branches such as `feature/...`, `fix/...`, or `docs/...`. Pull requests reference the issue they address, explain the change, and identify how the change was verified. Before merge, relevant automated checks must pass and at least one other team member must review substantive changes. Squash merging is preferred when it creates a clearer history.

A pull request is considered reviewable when it has a focused purpose, no unrelated changes, a clear verification note, and enough context that another team member can evaluate the decision without a private explanation. High impact decisions involving evaluation methodology, privacy, or architecture include a short rationale in the pull request or linked issue.

## 7.2 Issue Management

Issues are small enough for one person to complete in a reasonable work period and contain a clear completion condition. The team uses labels such as `documentation`, `frontend`, `backend`, `ml`, `testing`, `bug`, `meeting`, `research`, `blocked`, and `course-feedback`. Larger work is represented by a milestone or parent issue with smaller implementation issues beneath it.

Priority is recorded as `P0`, `P1`, or `P2`. P0 means the issue blocks a required milestone or creates a serious correctness or safety problem. P1 means required project work that should be completed in the active milestone. P2 means useful work that can move if higher risk tasks need time.

When an issue is closed, the closing comment or linked pull request states why it is complete. Meeting issues and course feedback issues are retained so important decisions remain traceable. GitHub milestones are used for major course deliverables, and a GitHub Project board may be used when parallel implementation work becomes large enough that a board improves visibility.

### 7.3 CI and CD

The repository already contains a GitHub Actions workflow that compiles LaTeX documentation and publishes project documentation. The team will keep this build green. As implementation begins, CI will also run formatters, linters, unit tests, and focused model or API tests that are suitable for automated execution. Expensive model evaluations will be separated from fast pull request checks so routine development does not wait on large experiments.

Deployment automation will be introduced after the product has a stable vertical slice. A deployment is not considered successful unless the same revision has passed its required checks. If a CI check is repeatedly flaky, the team will fix, quarantine with an issue and owner, or remove the unreliable check rather than normalize ignoring failures.

## 8 Project Decomposition and Scheduling

GitHub issues and milestones track the concrete project phases and target dates in Table 2, aligned with the SFWRENG 4G06 course schedule and the SRS Phase-In Plan (Issue #20).

Table 2: Project Schedule and Milestone Deadlines (2026–2027)

| **Target Date Window** | **Course Milestone (Week)** | **Primary Engineering & Documentation Deliverables** |
|---|---|---|
| Sept 15 – Sept 28, 2026 | **Week 04:** Problem Statement, Dev Plan & Team Charter Rev 0 | Establish repository, CI documentation build, initial problem framing, POC plan, and Team Charter. |
| Oct 1 – Oct 19, 2026 | **Week 06:** Req. Doc. and Hazard Analysis Rev 0 | Complete Meyer PEGS SRS (`FR1`–`FR14`, `NFR1`–`NFR10`, `SR1`–`SR9`), component FMEA (`H1`–`H10`), and October 7 TA consultation notes (Issue #17). |
| Oct 20 – Nov 2, 2026 | **Week 08:** V&V Plan Rev 0 | Build deterministic CLI benchmark runner (`FR12`, `FR14`), dataset split manifests, input validation gate (`FR2`), and V&V test specification. |
| Nov 3 – Nov 20, 2026 | **Weeks 10–12:** Design Doc Rev -1, CD-Fall Rev 0 & POC Demo | Benchmark candidate text (RoBERTa/DeBERTa) and image (ViT/CLIP + EXIF/C2PA) paths against naive baselines; calibrate abstention thresholds; deliver live POC demo and Fall ML Model Report. |
| Nov 21, 2026 – Jan 18, 2027 | **Week 16:** Design Document Rev 0 (MG/MIS) | Complete Chrome Manifest V3 extension UI, FastAPI request coordinator (`UUIDv4` isolation, 10.0 s timeout), consent modal, and 0.0 s ephemeral buffer cleanup. |
| Jan 19 – Feb 8, 2027 | **Weeks 18–19:** Revision 0 Demo + Interviews | Demonstrate full end-to-end browser extension + analysis service passing latency, privacy, cancellation, and concurrency checks. |
| Feb 9 – Mar 9, 2027 | **Week 22:** V&V Report Rev 0 & CD-Winter Rev 0 | Conduct usability study ($\geq$ 10 participants, $\geq$ 80% comprehension); execute full V&V regression, security fuzzing, and ESL/metadata fairness audits. |

| | | |
|---|---|---|
| Mar 10 – Apr 7, 2027 | **Weeks 24–26:** Final Demo (Mar 23), Final Docs (Apr 5) & EXPO (Apr 7) | Freeze Revision 1 documentation, package User Guide, and present final system at Capstone EXPO. |

The repository at github.com/richy1337/Verity is the authoritative project location.

# 9 Proof of Concept Demonstration Plan

This section defines the primary technical risk evaluated by our Proof of Concept (POC), our proactive pre-POC false-positive mitigations, our quantified baseline target, and the concrete candidate models and datasets for the November 2026 demonstration.

## 9.1 Primary Implementation Risk and Proactive Mitigations (Peer Issue #2)

The central technical risk is whether Verity can extract reliable, calibrated generation and manipulation signals from real web text and static images without suffering an unacceptably high False Positive Rate (FPR) on legitimate content (especially formal or non-native English (ESL) writing and re-compressed social-media photographs).

Rather than waiting for the POC to fail before addressing false positives (Peer Issue #2), the team will implement three **proactive pre-POC mitigations** during experiment construction:

1. **Calibrated Abstention Band (`inconclusive` Status):** Tune lower and upper decision thresholds $[\tau_{\text{low}}, \tau_{\text{high}}]$ on a validation split so borderline or conflicting inputs abstain as `inconclusive` rather than forcing a binary accusation (`FR4`, `SR1`).
2. **Separation of Provenance Metadata from Visual Forgery Signals:** Inspect `EXIF` and `C2PA` tags separately (`FR13`) so stripped metadata (common on social media) is reported as missing provenance rather than synthetic manipulation.
3. **Subgroup False-Positive Auditing:** Disaggregate benchmark FPR across ESL/formal prose slices and re-compressed authentic images (`FR14`, `NFR10`) prior to locking thresholds.

## 9.2 Proof of Concept Question and Quantified Baseline Target (Peer Issue #13)

The proof of concept will answer:

> Can the team build a reproducible analysis pipeline for text and static images that achieves a Macro-$F_1 \geq 0.80$ (`MIN_F1_SCORE`), a False Positive Rate FPR $\leq 10.0\%$ (`MAX_FPR`), and a Macro-$F_1$ improvement of at least $+\mathbf{0.10}$ (`MIN_BASELINE_DELTA`) over a documented naive baseline on held-out benchmark splits while exposing uncertainty, abstention coverage, and supporting evidence signals?

## 9.3 Concrete Candidate Detectors, Benchmarks, and Demonstration Scope (Issue #19)

Table 3 specifies the exact candidate detectors, held-out benchmark corpora, naive baselines, and fallback decision rules for the November 2026 POC demonstration.

Table 3: Concrete POC Detector Candidates, Baselines, Datasets, and Decision Rules

| Modality | Candidate Detector Path | Benchmark Dataset & Naive Baseline | Explicit POC Failure-to-Decision Rule |
|---|---|---|---|
| **Text Path** | Pretrained `RoBERTa` / `DeBERTa` open-weight AI-text detector paired with token perplexity and burstiness variance signals. | **Corpus:** Held-out split of **RAID** and **HC3** (including ESL and formal academic prose slices). **Baseline:** `TF-IDF` + Logistic Regression classifier and majority-class predictor. | If the candidate fails to exceed the `TF-IDF` baseline by $\geq +0.10$ Macro-$F_1$ or exceeds 10.0% FPR on ESL/formal slices after abstention tuning, restrict text output to descriptive burstiness/stylistic signals with mandatory `inconclusive` status on short/formal prose. |
| **Image Path** | Pretrained `ViT` / `CLIP` synthetic-image detector paired with deterministic `EXIF/C2PA` provenance parser and Error Level Analysis (ELA). | **Corpus:** Held-out split of **CIFAKE** and **GenImage** (including social-media re-compressed photos). **Baseline:** Shallow `ResNet-18` classifier and metadata-only heuristic. | If the visual classifier fails Macro-$F_1 \geq 0.80$ or $+0.10$ over baseline, narrow Revision 0 visual scope to provenance metadata + compression anomaly inspection or focus primary classification on the validated text modality with TA approval. |

During the live POC demonstration (executed on a team laptop running the local FastAPI service and CLI benchmark harness), **Calvin Salsali** will present the user workflow and uncertainty/evidence output, **Raymond Tao** will present the benchmark metrics and baseline comparison, and **Jaden Moore** and **Richard Han** will demonstrate live failure/abstention cases and reproducibility verification.

# 10 Engineering Tool Evaluation and Selection

The team will evaluate tools based on whether they reduce project risk and support reproducible engineering, not because they are common choices. The table records the current rationale, important limitations, and how the team plans to adapt each tool to Verity.

Table 4: Engineering Tool Evaluation

| Tool | Why it fits Verity | Limitations and planned adaptation |
|---|---|---|
| Git and GitHub | Provides version control, pull request review, issue traceability, milestones, and a public project history in one system. | GitHub activity can overvalue commit count and chat decisions can disappear. Contribution is therefore assessed using issues, reviews, tests, documentation, research, and code, while important decisions are copied into durable repository records. |

| Tool | Why it fits Verity | Limitations and planned adaptation |
|---|---|---|
| LaTeX, Make, GitHub Actions | The course templates are already LaTeX based and the repository already contains an automated documentation build. This gives reproducible PDFs and lets documentation changes be reviewed like code. | LaTeX has a higher editing cost than collaborative word processors and build failures can be cryptic. The team will keep document changes small, rely on the existing automated build, and use the course templates instead of custom document tooling. |
| Python and FastAPI | Python has strong machine learning and evaluation libraries. FastAPI provides a lightweight typed service boundary between analysis logic and the product interface. | A separate Python service adds deployment and interface complexity. The team will keep analysis contracts small and typed, and will not introduce multiple services unless the POC shows a clear need. |
| React and TypeScript | Fits an interactive browser experience and TypeScript improves interface checking between the user interface and analysis API. | Frontend frameworks can add unnecessary complexity. The team will favour a small component structure and will not adopt state management or UI libraries until a concrete need appears. |
| PyTorch and model libraries | Allow the team to evaluate existing models and detectors quickly instead of spending the capstone training a large model from scratch. | A pretrained model can be easy to run but unreliable on Verity's intended data. No model will be selected only because it is popular or reports a strong external score. Candidates must be evaluated using the team's own held out benchmark and failure analysis. |
| pytest and a TypeScript test runner | Support fast automated unit and integration tests in the two main language environments. | Test count and coverage can create false confidence. The team will prioritize tests tied to requirements, known failure modes, API contracts, and regressions, using coverage only as a diagnostic signal. |
| Ruff, Black, ESLint, Prettier | Automate style and basic static checks so review time is spent on design and correctness instead of formatting. | Automated linters do not evaluate architecture or semantic correctness. Passing lint is therefore necessary for code quality but never sufficient for merge approval. |

Tool choices will be revisited after the POC. A tool may be replaced if its limitations create measurable project risk or if another option provides a simpler path to the documented requirements.

# 11 Expected Technology

The initial implementation plan is consistent with the tool evaluation above. The project expects TypeScript with React for the browser interface, Python with FastAPI for the analysis service, and established machine learning libraries such as PyTorch and Hugging Face when their licences permit use. The team will prefer pretrained models or existing detection techniques during the POC. Training or fine tuning will be introduced only when evaluation evidence shows it is justified.

Persistence will remain minimal. A relational database may later support explicit features such as saved analyses, but the standard analysis path should be testable without retaining raw user submissions. Python tests will use `pytest`; TypeScript tests will use a suitable framework such as Vitest or Jest. Coverage tooling may be used diagnostically, but project acceptance will depend on meaningful tests and evaluation results rather than a percentage alone.

Git, GitHub, GitHub Issues, pull requests, milestones, GitHub Actions, and the repository's documentation build are part of the development environment from the beginning.

## 12 Use of AI and LLMs

The team may use AI assisted development tools for brainstorming, explaining unfamiliar APIs, generating small draft implementations, reviewing code, or improving documentation. AI generated material will not be merged solely because it appears plausible. Team members remain responsible for understanding code they submit, checking generated claims against primary sources, running tests, and reviewing diffs.

If an LLM is used inside Verity itself, it will not be treated as an authoritative truth source. Its output will be evaluated as one signal or explanation mechanism, with known limitations documented and with test cases designed to reveal confident but unsupported conclusions.

For technical decisions, the team will prefer official documentation, credible evaluation sources, reproducible experiments, and direct tests over uncited model output.

## 13 Coding Standard

The project will use language appropriate automated standards rather than rely on manual formatting preferences.

- TypeScript code will use strict typing where practical, consistent formatting, linting, clear component boundaries, and descriptive names.
- Python code will follow PEP 8 conventions, use type annotations for public interfaces where practical, and be automatically formatted and linted.
- Functions and modules should have one clear responsibility. Complex model evaluation logic should be separated from HTTP and user interface code.
- Public interfaces and nonobvious algorithms should be documented. Comments should explain why a decision exists rather than restate the code.
- New behaviour should include tests at the lowest useful level. Bug fixes should include a regression test when practical.
- Pull requests should be small enough to review meaningfully and should not mix unrelated refactoring with feature work without a clear reason.

# Appendix A: Generative AI Use Disclosure

In accordance with the SFWRENG 4G06 Generative AI policy, this appendix summarizes the AI tools used in preparing this Development Plan and the human verification steps completed by Team 23.

## AI Tools Used

- **Google Gemini 3.1 Pro** (accessed October 2026 via Antigravity IDE assistant) — used to format the LaTeX schedule and POC candidate tables, structure the 3-step Team Charter check-in protocol (Peer Issue #3), and synchronize the POC metrics with the companion SRS.
- **OpenAI ChatGPT (GPT-5)** (accessed September 2026) — used during initial September drafting to structure the workflow, tool evaluation, and coding standard sections.

## How and Where Used

- **Raymond Tao** used **Gemini 3.1 Pro** to format Table 3 (candidate text and image detectors, RAID/HC3 and CIFAKE/GenImage corpora, naive baselines, and +0.10 Macro-$F_1$ delta) in Section 9.
- **Jaden Moore** and **Richard Han** used **Gemini 3.1 Pro** to format the milestone schedule (Table 2) and the 3-step check-in resolution protocol in the Team Charter.
- **Calvin Salsali** used **Gemini 3.1 Pro** to review cross-document consistency across the Problem Statement, Development Plan, and SRS.

## Completed Verification of AI-Generated Content

All AI-assisted material was manually verified by the four members of Team 23 prior to committing this document (Issue #21):

1. **Repository, Tooling, and Schedule Audit (Verified by Jaden Moore & Richard Han):** Verified all repository links, MIT license references, GitHub Actions workflow descriptions, issue labels/priorities, and milestone dates against the live `richy1337/Verity` repository and the 2026–2027 SFWRENG 4G06 course calendar.
2. **POC Plan, Datasets, and Baseline Audit (Verified by Raymond Tao & Calvin Salsali):** Verified that candidate models (RoBERTa/DeBERTa, ViT/CLIP + EXIF/C2PA), naive baselines (TF-IDF + Logistic Regression, ResNet-18), benchmark datasets (RAID, HC3, CIFAKE, GenImage), and the +0.10 Macro-$F_1$ improvement threshold match the SRS (`NFR1`, Table 6, and Table 18).

# Appendix B: Reflection

1. **Why is it important to create a development plan prior to starting the project?**
2. **What are the advantages and disadvantages of using CI and CD?**
3. **What disagreements did your group have in this deliverable, if any, and how did you resolve them?**

## Team Synthesis

Creating the Development Plan before writing production code established a shared contract for how work is scoped in GitHub Issues, reviewed via pull requests, and validated through our November Proof of Concept (POC). Addressing feedback from Peer Review Team 12 (Issues #2, #3, #13) and TA Rebecca Di Filippo (Issues #19–#21) made our POC plan, schedule dates, and Team Charter check-in protocol substantially more concrete and actionable.

## Calvin Salsali

1. **Importance of a Development Plan:** Defining overlapping primary and secondary roles early prevented a situation where frontend work proceeds in isolation while ML benchmarking struggles with data formatting.
2. **CI/CD pros and cons:** Automated LaTeX PDF builds and TypeScript/ESLint checks catch formatting errors on every PR before teammates spend time reviewing, though we must keep PR checks fast so UI iteration is not slowed down.
3. **Disagreements and resolution:** We debated whether the initial product form factor should be a web app where users upload files or a browser extension. While a web app is simpler for a quick demo, we resolved the disagreement by committing to a Chrome Manifest V3 browser extension for the integrated product (because in-page selection removes user friction) while keeping the POC focused on the CLI benchmark harness and local FastAPI service.

## Raymond Tao

1. **Importance of a Development Plan:** Specifying our POC feasibility question, naive baselines (`TF-IDF`, `ResNet-18`), and +0.10 Macro-$F_1$ improvement threshold (Issue #13) up front prevents us from cherry-picking favourable examples later.
2. **CI/CD pros and cons:** Running deterministic unit tests on metric calculation and split-hash verification in CI protects against silent evaluation bugs, whereas running full GPU benchmark sweeps on every commit would be too slow and expensive, so we separated fast PR checks from scheduled/milestone benchmark runs.
3. **Disagreements and resolution:** We initially disagreed on whether to drop a modality immediately if false positives were high or try calibrating abstention bands first (Issue #2). We resolved this by adding proactive pre-POC mitigations (calibrated `inconclusive` abstention bands, separating stripped EXIF/C2PA metadata from pixel forgery, and ESL subgroup slicing) before triggering any modality-reduction decision rule.

## Jaden Moore

1. **Importance of a Development Plan:** Agreeing on branch naming, PR review rules, and concrete milestone dates (Table 2) ensures that no single member becomes a single point of failure.
2. **CI/CD pros and cons:** CI guarantees that code merged to `main` passes Ruff, Black, and `pytest` across clean environments, though flaky network tests can block merges if external APIs are not properly mocked.

3. **Disagreements and resolution:** During our Team Charter discussion, we debated how strictly to handle missed deadlines or check-ins (Issue #3). Some favoured informal chat reminders while others wanted clear accountability steps; we resolved this by adopting the structured 3-step check-in and pair-recovery protocol in the Team Charter below.

## Richard Han

1. **Importance of a Development Plan:** Establishing our testing and privacy standards before implementation ensures that zero-retention data handling and reproducibility are built into our architecture rather than bolted on at the end.

2. **CI/CD pros and cons:** Automated CI test suites give us repeatable evidence for our V&V Report, provided we prioritize requirement-driven tests (like zero-retention buffer checks and prompt-injection tests) over superficial line-coverage percentages.

3. **Disagreements and resolution:** We debated whether to include a persistent database for saving user history in our initial tech stack. We resolved this by making standard checks 100% stateless and ephemeral in memory so we eliminate privacy retention hazards from the outset.

# Appendix C: Team Charter

This Team Charter establishes Team 23's shared expectations for external goals, meeting attendance, work quality, interpersonal conduct, and our 3-step accountability check-in protocol.

## External Goals

The team's external goals are to produce a capstone project that is credible enough to demonstrate at the Capstone EXPO, useful enough to show to future employers or potential users, and technically deep enough that each member finishes the year with stronger experience in machine learning systems, software design, testing, and product decision making. The team also aims to maintain a repository and documentation set that another engineer could understand without relying on private team knowledge.

## Attendance

This subsection defines our attendance expectations, acceptable excuses, and emergency reassignment procedures.

### Expectations

Team members are expected to attend scheduled meetings on time and remain until agreed action items are assigned. A delay of more than 5 minutes should be communicated when possible. Missing more than two scheduled meetings in a four week period without prior notice triggers a direct team check-in following the 3-step resolution protocol below.

### Acceptable Excuse

Acceptable reasons for missing a meeting or deadline include illness, family or personal emergencies, unavoidable academic conflicts, interviews or recruiting obligations communicated in advance, and other circumstances that a reasonable teammate could not simply reschedule. Forgetting a meeting, repeatedly overcommitting without warning the team, or silently missing an agreed deadline is not acceptable behaviour.

### In Case of Emergency

A member facing an emergency should notify the group as soon as practical and identify any issue or deliverable that may be affected. The remaining members will reassign urgent work when necessary. When the member returns, the team will review the schedule and rebalance later work so a short term emergency does not permanently transfer that person's responsibilities to others.

## Accountability and Teamwork

This subsection defines our standards for deliverable quality, constructive communication, staying on track via our 3-step check-in protocol, and team building.

### Quality

Members should arrive at meetings having completed the preparation they committed to, or with a clear explanation of what blocked them. Code is not considered complete until it is reviewable, tested at an appropriate level, and integrated without breaking required checks. Documentation should be reviewed against the relevant course checklist and rubric at least 24 hours before a planned submission whenever the schedule allows.

### Attitude

The team critiques ideas and work products rather than people. Members are expected to ask questions early, share incomplete work when feedback would help, and respond constructively to review comments. Disagreements are first handled directly between the people involved. If that does not resolve the issue, the team discusses it together using project goals and available evidence rather than seniority or volume of opinion. Repeated interpersonal issues may be escalated to the TA. The repository's code of conduct applies to team interactions.

### Stay on Track and 3-Step Check-In Protocol (Peer Issue #3)

Each member must have visible current work in GitHub issues during an active project iteration. An issue more than 7 days overdue with no status update, two missed meetings in a 4-week window without notice, or two consecutive weekly cycles with no substantive contribution triggers a formal **Team Check-In**:

1. **Step 1 — During the Check-In (Root-Cause & Blocker Triage within 24 Hours):** The meeting chair and the team member hold a 15-minute diagnostic check-in to identify whether the delay stems from a technical blocker, unclear requirement scope, course/interview overload, or personal emergency, and document the blocker on the relevant GitHub Issue.

2. **Step 2 — Immediately After the Check-In (Scope Splitting, Pair Support & 48-Hour Checkpoint):** If the issue is too large or technically blocked, the team splits it into smaller sub-tasks, assigns a secondary role partner for a scheduled pair-programming session, and records a concrete 48-hour recovery checkpoint in the issue comments.

3. **Step 3 — Follow-Up & Escalation Criterion:** At the 48-hour checkpoint, if the recovery commitment is met, normal iteration resumes and workload balance is reviewed at the next weekly meeting. If two consecutive check-in recovery checkpoints are missed without an acceptable excuse, the variance is logged in the meeting minutes and escalated to the TA before it can impact a graded milestone.

Commit count alone is not used as a contribution measure. The team considers code, review, documentation, testing, research, design, meeting work, and other evidence recorded in the repository.

When the team reaches a milestone at least 24 hours before the internal deadline with required checks passing, the remaining time is used for review, testing, or helping teammates rather than adding unplanned scope unless the team agrees that the new work is low risk.

### Team Building

The team will make space for informal conversation before or after selected meetings and will acknowledge completed milestones rather than treating every meeting only as a status report. At major milestones, the team will briefly discuss what worked, what was frustrating, and one process change worth carrying forward.

### Decision Making

The preferred decision method is consensus informed by evidence. For ordinary reversible decisions, if discussion does not reach consensus within a reasonable amount of time, the team uses a majority vote and records the decision. High impact decisions involving project scope, architecture, privacy, or evaluation methodology include a short written rationale. If the team reaches a true deadlock on a high impact decision, it asks the TA, instructor, or appropriate advisor for guidance rather than allow the disagreement to stall the project.